

UNIVERSIDADE DO VALE DO PARAÍBA
FACULDADE DE ENGENHARIA, ARQUITETURA E URBANISMO
CURSO DE ENGENHARIA DA COMPUTAÇÃO

PROGAMES

Sistema Web de Locadora de Jogos Digitais e Board Games

Bruno Alexandre Holanda de Lima Silva
Victor Romero Siqueira dos Santos

Disciplina: Programação para Internet

São José dos Campos
2026

SUMÁRIO

1 TEMA ESCOLHIDO E OBJETIVO DO SISTEMA.....	3
2 TECNOLOGIAS UTILIZADAS	4
3 REGRAS DE NEGÓCIO	5
4 PERFIS DE ACESSO	6
5 ESTRUTURA MVC IMPLEMENTADA	7
6 INTERFACES IDAO, ISERVICE E ICONTROLLER.....	8
7 EXPLICAÇÃO DOS SERVICES	9
8 BANCO DE DADOS MYSQL	11
9 RELACIONAMENTOS DO BANCO.....	13
10 MONGODB E LOGS.....	13
11 UPLOAD DE IMAGENS E IMAGENS PADRÃO	14
12 EXPORTAÇÃO E IMPORTAÇÃO JSON.....	15
13 EXPORTAÇÃO XML DOS LOGS	16
14 RELATÓRIOS PDF.....	16
15 DASHBOARD E GRÁFICO CHART.JS	17
16 FLUXO PRINCIPAL DE USO DO SISTEMA.....	18
17 COMO EXECUTAR O PROJETO	19
18 ENDPOINTS/ROTAS DA API	20
19 CONSIDERAÇÕES FINAIS	23

1 TEMA ESCOLHIDO E OBJETIVO DO SISTEMA

O ProGames é um sistema web desenvolvido para a disciplina de Programação para Internet, com o objetivo de informatizar o funcionamento de uma locadora de jogos. O sistema abrange tanto jogos digitais (videogames de plataformas como PS5, Xbox Series X|S, Nintendo Switch e Nintendo Switch 2) quanto board games, permitindo que o estabelecimento controle seu catálogo, seu estoque e o ciclo de reservas e empréstimos realizados pelos clientes.

De forma geral, o sistema permite o gerenciamento de usuários, jogos, categorias, reservas, empréstimos, devoluções, estoque, relatórios, importação e exportação de dados, além do registro de logs de uso do sistema. O sistema foi pensado para atender dois perfis de uso, descritos com mais detalhes na seção 4 deste documento: o administrador, responsável pela gestão da locadora, e o usuário comum, que consulta o catálogo e solicita reservas.

De forma resumida, o ProGames contempla as seguintes funcionalidades principais:

- Cadastro e autenticação de usuários, com distinção entre administrador e usuário comum;
- Cadastro de categorias e de jogos, com upload opcional de imagem de capa;
- Catálogo de jogos disponível para consulta pelo usuário comum, com filtros de busca;
- Criação de reservas pelo usuário comum e de empréstimos diretamente pelo administrador;
- Controle automático de estoque ao reservar, emprestar, devolver ou cancelar uma solicitação;
- Devolução de jogos, com atualização do status do empréstimo;
- Dashboard administrativo com indicadores e gráfico dos jogos mais alugados;
- Geração de relatórios em PDF de jogos e de empréstimos;
- Exportação e importação de dados em formato JSON;
- Registro de logs de uso do sistema e exportação desses logs em formato XML.

2 TECNOLOGIAS UTILIZADAS

O ProGames foi construído como uma aplicação web full stack, com o backend e o frontend organizados em pastas separadas dentro do mesmo projeto. As tecnologias e bibliotecas utilizadas em cada parte do sistema são apresentadas a seguir.

Backend

- Node.js, como ambiente de execução do servidor;
- Express, como framework para definição de rotas e middlewares da API REST.

Frontend

- HTML5, para a estrutura das páginas;
- CSS3, para a estilização das telas;
- JavaScript, para o consumo da API e a manipulação dinâmica das páginas;
- Bootstrap, como framework de componentes visuais e layout responsivo.

Bancos de dados

- MySQL, utilizado para os dados principais do sistema (usuários, categorias, jogos e empréstimos);
- MongoDB, utilizado para o registro dos logs de uso do sistema.

Bibliotecas e recursos complementares

Biblioteca/recurso	Finalidade no projeto
jsonwebtoken (JWT)	Geração e validação do token de autenticação
bcryptjs	Geração do hash das senhas dos usuários
mysql2	Conexão e execução de consultas no banco MySQL
mongodb	Conexão e operações na coleção de logs do MongoDB
multer	Upload das imagens de capa dos jogos
chart.js	Geração dos gráficos do dashboard administrativo, no frontend
pdfkit	Geração dos relatórios em PDF, no backend
xml2js	Conversão dos logs para o formato XML na exportação

3 REGRAS DE NEGÓCIO

O acesso ao sistema é controlado por autenticação baseada em login e senha, com geração de um token JWT (JSON Web Token) no momento do login. Esse token é exigido nas

requisições subsequentes, no cabeçalho Authorization, e é validado por um middleware de autenticação antes que a requisição alcance os controllers. Não possuindo token válido, a requisição é rejeitada antes de chegar à camada de regras de negócio.

Cada jogo cadastrado no sistema possui um valor de diária (campo valor_aluguel). O valor de um empréstimo ou reserva depende da quantidade de dias solicitada, calculada pela diferença entre a data de retirada e a data prevista de devolução informadas. Um empréstimo pode conter vários jogos ao mesmo tempo, cada um com sua quantidade; esse relacionamento entre empréstimos e jogos é de muitos para muitos, resolvido por meio da tabela itens_emprestimo.

A partir de três diárias, o sistema aplica um desconto de R\$ 10,00 por item, ou seja, por unidade de jogo presente no empréstimo. O valor total considera, portanto, o valor da diária de cada jogo, a quantidade de jogos solicitados, o número de dias do empréstimo e o desconto aplicável.

Antes de confirmar uma reserva ou um empréstimo, o sistema verifica o estoque disponível do jogo. Caso o estoque seja insuficiente, a solicitação é rejeitada. Ao ser confirmada, o estoque é reduzido conforme a quantidade solicitada. Esse controle também ocorre no caminho inverso: ao registrar a devolução de um empréstimo, ou quando uma reserva é cancelada, o estoque correspondente retorna automaticamente, evitando inconsistências entre o número de exemplares físicos e o que está registrado como disponível.

Cada empréstimo possui um status que reflete seu andamento: pendente (reserva aguardando confirmação), aprovado, retirado, devolvido, cancelado e atrasado. O sistema não realiza cobrança de multa por atraso; o controle do atraso é feito exclusivamente por meio do status atrasado, que não é gravado diretamente no banco de dados, mas calculado de forma automática nas consultas sempre que a data prevista de devolução já passou e o empréstimo ainda está em aberto (ou seja, ainda não foi devolvido nem cancelado).

Todas as ações relevantes realizadas no sistema — autenticação, cadastros, atualizações, remoções, devoluções, geração de relatórios e erros — são registradas como logs em uma base separada (MongoDB), o que permite manter um histórico de auditoria independente das tabelas operacionais do MySQL.

4 PERFIS DE ACESSO

Existem dois tipos de usuário definidos na tabela usuarios: admin e usuario. Essa distinção é usada por um middleware de permissão, responsável por liberar ou bloquear o acesso a determinadas rotas conforme o perfil do usuário autenticado.

Administrador

- Acessa o painel administrativo do sistema;
- Gerencia jogos, categorias, usuários, empréstimos e reservas;
- Pode criar novos usuários, incluindo outros administradores;
- Pode cadastrar, editar e excluir jogos e categorias;
- Pode aprovar, editar, cancelar e devolver empréstimos e reservas;
- Pode gerar os relatórios em PDF e exportar os logs do sistema;
- Pode exportar e importar dados em formato JSON;
- Acessa o dashboard administrativo, com os indicadores da locadora.

Usuário comum

- Acessa o catálogo de jogos disponíveis na locadora;
- Filtra os jogos por título, plataforma, categoria e tipo (videogame ou board game);
- Visualiza os detalhes de cada jogo do catálogo;
- Solicita empréstimos e reservas, escolhendo jogos, datas e quantidades;
- Acompanha suas reservas em aberto na tela Minhas Reservas;
- Consulta o histórico de jogos já devolvidos ou cancelados na tela Histórico;
- Visualiza e mantém os dados do próprio perfil;
- Não tem acesso ao cadastro, edição ou exclusão de jogos e categorias, restritos ao administrador.

A aprovação, a edição e a devolução de um empréstimo ou reserva são operações concentradas no painel administrativo. O usuário comum acompanha o andamento de suas solicitações pelas telas Minhas Reservas e Histórico, mas a gestão do ciclo de vida do empréstimo (aprovar, editar, devolver) é responsabilidade do administrador. A interface do usuário comum oferece, ainda assim, a opção de cancelar uma reserva enquanto ela estiver pendente, sendo essa a única ação de gestão disponível diretamente para esse perfil.

5 ESTRUTURA MVC IMPLEMENTADA

O backend do ProGames segue uma organização baseada no padrão MVC, complementada por uma camada de Service e pelo padrão DAO (Data Access Object), de forma a separar claramente as responsabilidades de cada parte do sistema. A estrutura de pastas do backend é organizada da seguinte forma: config, controllers, daos, interfaces, middlewares, models, routes, services e utils, além da pasta uploads para armazenamento das imagens de capa enviadas pelo administrador.

A responsabilidade de cada camada pode ser descrita da seguinte forma:

- Routes/Routers: definem os endpoints da API e os métodos HTTP aceitos, associando cada rota ao respectivo controller e aos middlewares necessários (autenticação, permissão, validação de corpo, upload de arquivo);
- Middlewares: interceptam a requisição antes ou depois do controller, sendo usados para autenticação via JWT, verificação de permissão por perfil, validação do corpo das requisições, tratamento de upload de arquivos com Multer, registro de logs e tratamento centralizado de erros;
- Controllers: recebem as requisições já validadas e autenticadas, extraem os dados necessários (parâmetros, corpo da requisição, arquivo enviado, usuário autenticado), chamam o respectivo Service e devolvem a resposta ao cliente em formato JSON padronizado;
- Services: concentram as regras de negócio do sistema, como cálculo de valores, validação de estoque, controle de permissões específicas de cada operação e orquestração das chamadas aos DAOs;
- DAOs: são responsáveis exclusivamente pelo acesso ao banco de dados, executando as consultas SQL (ou operações no MongoDB, no caso dos logs) por meio de prepared statements, sem conter regras de negócio;
- Models: representam as entidades manipuladas pelo sistema. No caso dos logs, por exemplo, o model LogSistema define os campos que compõem um registro de log antes de ele ser persistido no MongoDB;
- Config: concentra a inicialização das conexões com o MySQL e com o MongoDB, além das variáveis de ambiente utilizadas pela aplicação;

- Frontend: mantido separado do backend, composto por páginas HTML, arquivos de estilo CSS e scripts JavaScript que consomem a API REST exposta pelo backend.

Essa separação permite que o frontend seja tratado como um cliente da API, sem acesso direto ao banco de dados, e que cada camada do backend tenha uma responsabilidade única e bem definida.

6 INTERFACES IDAO, ISERVICE E ICONTROLLER

Para padronizar o comportamento esperado das camadas de acesso a dados, regras de negócio e controle de requisições, o projeto define três interfaces na pasta backend/src/interfaces: IDAO, IService e IController. Em JavaScript não existe o conceito de interface da forma como ocorre em linguagens como Java ou C#, de modo que essas classes foram implementadas como classes base que declaram os métodos esperados (listar, buscarPorId, criar, atualizar e remover), lançando um erro caso uma subclasse não sobrescreva o método correspondente.

Essa abordagem cumpre um papel semelhante ao de uma interface: ela documenta o contrato que cada DAO, Service ou Controller deve seguir, e obriga, em tempo de execução, que as classes concretas implementem os métodos previstos. Isso facilita a manutenção do projeto, pois qualquer novo DAO, Service ou Controller criado para uma nova entidade segue o mesmo conjunto mínimo de operações já utilizado nos demais módulos do sistema.

As classes que estendem cada uma dessas interfaces são as seguintes:

Interface	Classes que a implementam
IDAO	UsuarioDAO, CategoriaDAO, JogoDAO, EmprestimoDAO, LogDAO
IService	JogoService, CategoriaService, EmprestimoService, LogService (UsuarioService e AuthService seguem o mesmo padrão de métodos, sem herdar diretamente de IService)
IController	Os controllers do projeto seguem a mesma assinatura de métodos definida em IController (listar, buscarPorId, criar, atualizar, remover)

Nem todos os DAOs e Services do projeto possuem exatamente os cinco métodos da interface. No LogDAO, por exemplo, os métodos atualizar e remover foram intencionalmente sobrescritos para lançar um erro, já que registros de log não devem ser alterados ou excluídos, preservando a integridade do histórico de auditoria. Já o DashboardDAO e o DashboardService, por terem uma finalidade exclusivamente de leitura e agregação de dados para indicadores, não seguem o mesmo contrato de CRUD das demais entidades.

7 EXPLICAÇÃO DOS SERVICES

A camada de Service concentra as regras de negócio do ProGames, evitando que esse tipo de lógica fique espalhada nos controllers ou misturada às consultas SQL dos DAOs. Cada Service recebe os dados já tratados pelo controller, aplica as validações e cálculos pertinentes, chama o DAO correspondente para persistir ou consultar os dados e, quando necessário, registra um log da operação por meio do LogService.

AuthService

Responsável pela autenticação dos usuários. Recebe o e-mail e a senha informados no login, busca o usuário pelo e-mail por meio do UsuarioDAO e compara a senha informada com o hash armazenado utilizando bcryptjs. Em caso de sucesso, gera um token JWT contendo o identificador, o nome, o e-mail e o tipo do usuário. Tanto as tentativas de login bem-sucedidas quanto as que falham são registradas como log, assim como a operação de logout.

JogoService

Implementa as regras relacionadas ao cadastro de jogos: validação da existência da categoria informada antes de criar ou atualizar um jogo, conversão dos campos numéricos (valor da diária e estoque), associação do caminho da imagem enviada via upload e ajuste manual de estoque, utilizado em situações administrativas pontuais.

CategoriaService

Trata o CRUD de categorias de jogos, que são utilizadas tanto para organizar o catálogo quanto como chave estrangeira na tabela de jogos.

UsuarioService

Responsável pelo CRUD de usuários do sistema, incluindo a geração do hash da senha antes de persistir o registro no banco de dados, de forma que a senha em texto puro nunca seja armazenada na tabela usuarios.

EmprestimoService

Concentra as regras relacionadas a empréstimos e reservas. Valida se a lista de itens informada é válida (cada item precisa de um jogo e de uma quantidade maior que zero), delega ao EmprestimoDAO o cálculo de diárias, desconto e valor total dentro de uma transação, e

oferece operações específicas como criação de empréstimo direto pelo administrador, criação de reserva pelo usuário comum, devolução, atualização e remoção.

RelatorioService

Gera os relatórios em PDF do sistema, descritos com mais detalhes na seção 14 deste documento. Busca os dados necessários junto ao JogoDAO ou ao EmpréstimoDAO e monta o documento PDF com cabeçalho, tabela de dados e um resumo final.

DashboardService

Reúne, em uma única chamada, os indicadores utilizados no dashboard administrativo: o resumo geral do sistema, o ranking de jogos mais alugados e a distribuição de empréstimos por status.

ExportacaoService

Implementa a exportação e a importação de dados em formato JSON para as entidades usuários, categorias, jogos e empréstimos, descritas com mais detalhes na seção 12 deste documento.

LogService

Responsável por registrar, listar e exportar os logs do sistema. É chamado pelos demais Services e por um middleware sempre que uma ação relevante precisa ser registrada. Também concentra a lógica de exportação dos logs para o formato XML, descrita na seção 13 deste documento.

8 BANCO DE DADOS MYSQL

O banco de dados relacional do ProGames é o MySQL, chamado `locacao_jogos`, criado por um único script, `database/banco.sql`. Esse script cria as tabelas, as chaves estrangeiras, os índices e os dados iniciais de teste, incluindo dois usuários (um administrador e um usuário comum), vinte categorias e um conjunto de cento e vinte jogos cadastrados. O banco é composto por cinco tabelas principais: `usuarios`, `categorias`, `jogos`, `emprestimos` e `itens_emprestimo`.

O arquivo de consultas do projeto demonstra o uso de comandos `SELECT`, `INSERT`, `UPDATE` e `DELETE`, com cláusulas `WHERE` para filtros e `JOIN` para consultas que combinam

dados de mais de uma tabela, além do uso de chaves estrangeiras para manter a integridade entre as tabelas relacionadas.

Tabela usuarios

Armazena os dados de quem acessa o sistema, com os seguintes campos principais:

Campo	Descrição
id	Identificador único, chave primária
nome	Nome do usuário
email	E-mail utilizado no login, com restrição de unicidade
senha	Hash da senha, gerado com bcrypt
telefone	Telefone de contato
tipo_usuario	Enumeração admin ou usuario, define o perfil de acesso
created_at	Data de criação do registro

Tabela categorias

Organiza os jogos por gênero ou estilo, com os seguintes campos:

Campo	Descrição
id	Identificador único, chave primária
nome	Nome da categoria, com restrição de unicidade
descricao	Descrição livre da categoria

Tabela jogos

Armazena o catálogo de jogos disponíveis para locação:

Campo	Descrição
id	Identificador único, chave primária
titulo	Título do jogo
descricao	Descrição livre do jogo
categoria_id	Chave estrangeira para categorias
tipo_jogo	Enumeração videogame ou boardgame
plataforma	Plataforma do jogo (PS5, Xbox Series X S, Nintendo Switch, Nintendo Switch 2 ou Board Game)
valor_aluguel	Valor da diária de locação
estoque	Quantidade de exemplares disponíveis
imagem	Caminho da imagem de capa enviada pelo administrador

Campo	Descrição
created_at	Data de criação do registro

Tabela empréstimos

Representa cada solicitação de empréstimo ou reserva feita por um usuário:

Campo	Descrição
id	Identificador único, chave primária
usuario_id	Chave estrangeira para usuarios
data_emprestimo	Data de retirada do empréstimo
data_prevista_devolucao	Data limite prevista para devolução
data_devolucao_real	Data e hora em que a devolução efetivamente ocorreu
status	pendente, aprovado, retirado, devolvido, cancelado ou atrasado
dias_aluguel	Quantidade de diárias calculada para o empréstimo
desconto	Valor de desconto aplicado, por item, a partir de três diárias
valor_total	Valor total cobrado pelo empréstimo, já considerando o desconto
observacoes	Observações livres sobre a solicitação
created_at	Data de criação do registro
updated_at	Data da última atualização do registro

O sistema não possui um campo de multa na tabela empréstimos. O controle de atraso é feito exclusivamente pelo status atrasado, calculado de forma automática conforme descrito na seção 3 deste documento, sem qualquer cobrança adicional sobre o valor já registrado no empréstimo.

Tabela itens_emprestimo

Faz a ligação entre um empréstimo e os jogos que o compõem, permitindo que um único empréstimo contenha mais de um jogo. Possui os campos id, emprestimo_id (chave estrangeira para empréstimos), jogo_id (chave estrangeira para jogos), quantidade e valor_unitario, este último armazenado no momento da criação do empréstimo para preservar o valor cobrado mesmo que o preço do jogo seja alterado posteriormente.

9 RELACIONAMENTOS DO BANCO

Os relacionamentos entre as tabelas do banco de dados são os seguintes:

- usuarios 1:N empréstimos — um usuário pode realizar diversos empréstimos, mas cada empréstimo pertence a um único usuário;

- categorias 1:N jogos — uma categoria pode estar associada a diversos jogos, mas cada jogo pertence a uma única categoria;
- empréstimos N:N jogos, por meio de itens_emprestimo — um empréstimo pode conter diversos jogos e um mesmo jogo pode aparecer em diversos empréstimos ao longo do tempo, sendo essa relação de muitos-para-muitos resolvida pela tabela associativa itens_emprestimo.

Além das chaves estrangeiras, o script de criação do banco define índices sobre os campos mais utilizados em filtros e buscas, como categoria_id, titulo, tipo_jogo e plataforma na tabela jogos, e usuario_id, status e data_prevista_devolucao na tabela empréstimos, com o objetivo de melhorar o desempenho das consultas mais frequentes do sistema.

10 MONGODB E LOGS

Além do banco relacional MySQL, o ProGames utiliza o MongoDB para registrar os logs de uso do sistema, em uma coleção chamada logs, dentro do banco locacao_jogos_logs. Os logs registram ações importantes realizadas no sistema, como login, criação, atualização, remoção, empréstimos, devoluções, geração de relatórios e erros. Cada documento da coleção representa um evento ocorrido e possui os seguintes campos:

Campo	Descrição
usuario	E-mail do usuário que realizou a ação, ou 'anonimo' quando não autenticado
endpoint	Rota da API acessada
metodo	Método HTTP utilizado na requisição
timestamp	Data e hora em que o evento ocorreu
status_code	Código de status HTTP retornado pela requisição
acao	Identificador da ação realizada, como CRIAR_JOGO ou LOGIN_SUCESSO
ip	Endereço IP de origem da requisição
detalhes	Objeto com informações adicionais sobre o evento, como dados antes e depois de uma alteração

Os registros são gerados de duas formas: por um middleware de log, que registra automaticamente toda requisição finalizada, e diretamente pelos Services, sempre que uma operação de cadastro, atualização, remoção, devolução ou autenticação é executada, complementando o registro genérico do middleware com detalhes específicos da operação. A listagem de logs no painel administrativo apresenta os registros mais recentes primeiro,

permitindo filtros por usuário, ação e período, de forma que o administrador consiga localizar com facilidade as ações realizadas no sistema.

O uso do MongoDB para essa finalidade se justifica por se tratar de dados de auditoria, que não possuem uma estrutura fixa e podem variar bastante de um evento para outro, já que o campo detalhes armazena informações diferentes dependendo da ação registrada. Um banco de documentos permite gravar esse tipo de informação sem a necessidade de definir previamente um esquema rígido de colunas, o que seria mais difícil de manter caso os logs fossem armazenados em uma tabela do MySQL. Além disso, manter os logs em um banco separado do MySQL evita que o alto volume de registros de auditoria impacte o desempenho das tabelas operacionais do sistema.

Para que a aplicação continue funcionando mesmo que o MongoDB não esteja disponível no momento, o LogDAO possui um mecanismo de fallback: caso a gravação no MongoDB falhe, o log é salvo em um arquivo local no formato JSON Lines (backend/logs/logs_fallback.jsonl). Esse mecanismo evita que a indisponibilidade do MongoDB interrompa as demais funcionalidades do sistema, embora o uso do MongoDB seja o caminho esperado para o registro correto dos logs.

11 UPLOAD DE IMAGENS E IMAGENS PADRÃO

No cadastro ou na edição de um jogo, o administrador pode enviar uma imagem de capa correspondente ao título. Esse upload é processado no backend pela biblioteca Multer, que recebe o arquivo enviado na requisição e o salva em disco, na pasta backend/uploads. O caminho do arquivo salvo é então armazenado no campo imagem da tabela jogos, e é esse caminho que o frontend utiliza para exibir a capa do jogo no catálogo.

Quando o administrador não envia uma imagem própria para o jogo, o frontend exibe automaticamente uma imagem padrão, definida de acordo com a plataforma cadastrada. As imagens padrão ficam armazenadas no próprio frontend, na pasta assets/img, e existe uma imagem para cada uma das plataformas suportadas pelo sistema:

- PS5;
- Xbox Series X|S;
- Nintendo Switch;
- Nintendo Switch 2;

- Board Game.

Esse mecanismo evita que o catálogo fique visualmente vazio nos casos em que o administrador ainda não cadastrou uma capa específica para o jogo, mantendo a listagem de jogos consistente mesmo antes do envio de imagens próprias.

12 EXPORTAÇÃO E IMPORTAÇÃO JSON

O ProGames permite exportar e importar dados em formato JSON para as entidades usuários, categorias, jogos e empréstimos. Essa é uma funcionalidade administrativa, disponível apenas para o perfil admin, que pode ser útil para situações como backup dos dados cadastrados, migração entre ambientes, preparação de dados de teste ou carga inicial de informações no sistema.

Na exportação, o ExportacaoService busca os registros da entidade solicitada por meio do ExportacaoDAO e devolve a lista em formato JSON, que pode ser salva pelo administrador como arquivo. Na importação, o sistema lê o arquivo JSON enviado, valida se o conteúdo corresponde a uma lista de registros e reaproveita os próprios Services de cada entidade (UsuarioService, CategoriaService, JogoService e EmprestimoService) para criar os registros, um a um, mantendo as mesmas validações e os mesmos logs já existentes para cada tipo de cadastro.

As entidades aceitas pelas rotas de exportação e importação são usuarios, categorias, jogos e empréstimos, conforme implementado no ExportacaoDAO e no ExportacaoService.

13 EXPORTAÇÃO XML DOS LOGS

O sistema permite exportar os logs armazenados no MongoDB em formato XML, por meio da rota GET /api/logs/exportar/xml, restrita ao perfil administrador. Essa exportação é realizada pelo LogService, utilizando a biblioteca xml2js para converter a lista de logs em um documento XML estruturado.

O documento gerado segue um formato com uma raiz logs, contendo uma lista de elementos evento, cada um numerado por um atributo id. Cada elemento evento apresenta os seguintes dados do log correspondente: usuario, acao, descricao (extraída do campo detalhes ou do endpoint, quando não há uma descrição específica), data_hora (convertida para o formato ISO), tipo_evento (uma classificação derivada da ação registrada, como autenticacao, cadastro, erro ou acesso), endpoint, metodo, status_code, ip_origem e detalhes (o conteúdo do campo

detalhes convertido para texto em formato JSON). A estrutura geral do documento pode ser resumida da seguinte forma:

```
<?xml version="1.0" encoding="UTF-8"?>
<logs>
  <evento id="1">
    <usuario>admin@locadora.com</usuario>
    <acao>LOGIN_SUCESSO</acao>
    <descricao>...</descricao>
    <data_hora>2026-05-12T14:20:00.000Z</data_hora>
    <tipo_evento>autenticacao</tipo_evento>
    <endpoint>/api/auth/login</endpoint>
    <metodo>POST</metodo>
    <status_code>200</status_code>
    <ip_origem>127.0.0.1</ip_origem>
    <detalhes>{...}</detalhes>
  </evento>
</logs>
```

Esse formato é útil porque o XML é um padrão amplamente aceito para troca de dados entre sistemas diferentes, o que facilita a interoperabilidade caso os logs precisem ser consumidos por outra aplicação, por uma ferramenta externa de auditoria ou por um processo de análise posterior. Diferentemente da exportação em JSON, utilizada para as entidades operacionais do sistema, a exportação em XML foi reservada especificamente para os logs, reforçando seu uso como mecanismo de auditoria e registro histórico do sistema.

14 RELATÓRIOS PDF

O ProGames gera relatórios em PDF por meio da biblioteca PDFKit, utilizada no RelatorioService. Essa biblioteca permite montar o documento de forma programática, desenhando texto, tabelas e o controle de quebra de página diretamente no backend, sem depender de um template HTML convertido posteriormente para PDF. O documento é construído em memória e devolvido como um buffer, que é enviado ao navegador do administrador como resposta da requisição. Os dois relatórios disponibilizados utilizam dados reais consultados no banco MySQL no momento da geração, e ambos são restritos ao perfil administrador.

Relatório de jogos (GET /api/relatorios/jogos/pdf)

Lista o acervo de jogos cadastrados de acordo com os filtros informados na requisição, apresentando título, categoria, tipo de jogo, plataforma, estoque disponível e valor da diária de cada item. Ao final da tabela, é exibido um resumo com o total de jogos listados no relatório.

Relatório de empréstimos (GET /api/relatorios/emprestimos/pdf)

Lista os empréstimos cadastrados, também conforme os filtros aplicados, apresentando identificador, usuário responsável, jogos envolvidos no empréstimo, quantidade de diárias, status, valor de desconto aplicado e valor total do empréstimo. No resumo final, são apresentados o total de empréstimos listados e o somatório do valor financeiro envolvido nesses empréstimos.

Em ambos os relatórios, o documento inicia com um cabeçalho contendo o título do relatório, a data e hora de geração e o nome (ou e-mail) do usuário administrador que solicitou a emissão. A montagem da tabela é feita linha a linha, com larguras de coluna fixas para cada relatório, e o conteúdo das células é truncado e finalizado com reticências quando excede a largura disponível, evitando que textos longos quebrem o layout. Quando o conteúdo excede o espaço de uma página, o RelatorioService insere automaticamente uma nova página e repete o cabeçalho da tabela, garantindo a legibilidade de relatórios mais extensos.

15 DASHBOARD E GRÁFICO CHART.JS

O dashboard é uma área exclusiva do administrador, que apresenta indicadores gerais sobre o funcionamento da locadora. No frontend, esses indicadores são exibidos com o auxílio da biblioteca Chart.js, responsável por desenhar os gráficos a partir dos dados recebidos da API.

Os dados exibidos no dashboard são obtidos da rota GET /api/dashboard/resumo, que devolve as informações já calculadas pelo DashboardService a partir de consultas SQL realizadas pelo DashboardDAO diretamente no banco MySQL. Os indicadores retornados por essa rota são organizados em três grupos:

- **Resumo geral:** total de usuários cadastrados, total de jogos cadastrados, soma do estoque disponível, quantidade de empréstimos ativos (nos status pendente, aprovado ou retirado), quantidade de empréstimos atrasados (empréstimos ativos cuja data prevista de devolução já passou) e o total arrecadado, somando o valor total dos empréstimos já devolvidos;
- **Jogos mais alugados:** ranking dos oito jogos com maior quantidade total de unidades alugadas, calculado a partir da soma das quantidades registradas na tabela itens_emprestimo;

- Empréstimos por status: quantidade de empréstimos agrupados por status (pendente, aprovado, retirado, devolvido, cancelado ou atrasado), utilizada para um gráfico de distribuição que mostra a proporção de empréstimos em cada situação.

Com base nesses indicadores, o administrador consegue visualizar rapidamente a situação geral da locadora, identificar quais jogos têm maior saída e acompanhar a quantidade de empréstimos pendentes de devolução, sem precisar consultar diretamente as tabelas do banco de dados.

16 FLUXO PRINCIPAL DE USO DO SISTEMA

De forma a ilustrar como as funcionalidades descritas nas seções anteriores se conectam na prática, esta seção apresenta o fluxo típico de uso do ProGames, do cadastro inicial de um usuário até o registro dos logs gerados pelas operações realizadas.

- O administrador realiza login no sistema, informando e-mail e senha;
- O administrador acessa o painel administrativo, que reúne as opções de gerenciamento do sistema;
- O administrador cadastra um usuário comum, caso este ainda não possua acesso ao sistema;
- O usuário comum realiza login com as credenciais cadastradas;
- O usuário acessa o catálogo de jogos, podendo filtrar por título, plataforma, categoria e tipo;
- O usuário escolhe um ou mais jogos e solicita um empréstimo ou reserva, informando a data de retirada e a data prevista de devolução;
- O sistema valida o estoque disponível dos jogos escolhidos e as datas informadas, e calcula o subtotal, o desconto aplicável e o valor total previsto para a solicitação;
- O administrador visualiza a solicitação na área de empréstimos e reservas do painel administrativo, que mostra quais jogos foram alugados ou reservados em cada solicitação;
- O administrador aprova, edita ou, se necessário, cancela a reserva;
- O administrador também pode criar empréstimos diretamente, sem que exista uma reserva prévia do usuário;

- O sistema atualiza o estoque dos jogos envolvidos, reduzindo a quantidade disponível ao confirmar a solicitação e restaurando o estoque em caso de devolução ou cancelamento;
- O administrador pode gerar relatórios em PDF de jogos ou de empréstimos a qualquer momento;
- Ao longo de todo o fluxo, o sistema registra logs no MongoDB para as ações relevantes realizadas, como login, cadastro, aprovação, devolução e geração de relatórios.

O usuário comum acompanha o andamento de suas solicitações pelas telas Minhas Reservas, que lista as reservas em aberto, e Histórico, que apresenta os jogos já devolvidos ou cancelados. A gestão propriamente dita do ciclo de vida do empréstimo, no entanto, permanece concentrada no painel administrativo.

17 COMO EXECUTAR O PROJETO

Para executar o ProGames em ambiente de desenvolvimento, é necessário possuir o Node.js instalado, além do MySQL (disponibilizado, neste projeto, por meio do XAMPP) e, idealmente, do MongoDB em execução para o registro correto dos logs. O passo a passo de instalação e execução é o seguinte:

1. Instalar o Node.js, em sua versão LTS mais recente, disponível no site oficial do projeto;
2. Instalar o XAMPP, que será utilizado para disponibilizar o servidor MySQL local (o módulo Apache do XAMPP não é necessário para a execução da aplicação, apenas o MySQL);
3. Iniciar o serviço MySQL pelo painel de controle do XAMPP;
4. Abrir o phpMyAdmin (geralmente em <http://localhost/phpmyadmin>) ou o MySQL Workbench;
5. Executar o arquivo database/banco.sql, responsável por criar o banco locacao_jogos, suas tabelas, índices, chaves estrangeiras e os dados iniciais de teste;
6. Criar ou ajustar o arquivo .env do projeto com as configurações de conexão ao banco de dados (host, usuário e senha do MySQL e, se aplicável, a string de conexão do MongoDB);

7. Abrir um terminal na pasta raiz do projeto e executar o comando `npm install`, para instalar as dependências (Express, mysql2, mongodb, jsonwebtoken, bcryptjs, multer, pdfkit, xml2js, entre outras);
8. Executar o comando `npm start`, ou `npm run dev` caso o projeto possua esse script configurado para desenvolvimento com reinício automático;
9. Acessar o endereço `http://localhost:3000` no navegador para utilizar o sistema;
10. Entrar com um dos usuários de teste cadastrados pelo script do banco de dados.

Os usuários de teste cadastrados pelo script do banco de dados são os seguintes:

Perfil	E-mail	Senha
Administrador	admin@locadora.com	123456
Usuário comum	usuario@locadora.com	123456

18 ENDPOINTS/ROTAS DA API

Todas as rotas da API são acessadas a partir de `http://localhost:3000`, com o prefixo `/api` antes de cada endpoint, conforme apresentado nas tabelas a seguir. Com exceção da rota de login, todas as demais exigem o envio de um token JWT válido no cabeçalho `Authorization`, no formato `Bearer TOKEN_JWT`. As tabelas a seguir apresentam as rotas organizadas por módulo, indicando o método HTTP, o endpoint completo, uma descrição curta, a exigência de autenticação e o perfil de acesso necessário.

Auth

Método	Endpoint	Descrição	Requer login	Perfil
POST	<code>/api/auth/login</code>	Autentica o usuário e gera o token JWT	Não	Público
POST	<code>/api/auth/logout</code>	Registra o logout do usuário autenticado	Sim	Admin e usuário
GET	<code>/api/auth/me</code>	Retorna os dados do usuário autenticado	Sim	Admin e usuário

Usuários

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/usuarios	Lista os usuários cadastrados	Sim	Admin
GET	/api/usuarios/:id	Busca um usuário pelo identificador	Sim	Admin
POST	/api/usuarios	Cadastra um novo usuário	Sim	Admin
PUT	/api/usuarios/:id	Atualiza os dados de um usuário	Sim	Admin
DELETE	/api/usuarios/:id	Remove um usuário	Sim	Admin

Categorias

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/categorias	Lista as categorias cadastradas	Sim	Admin e usuário
GET	/api/categorias/:id	Busca uma categoria pelo identificador	Sim	Admin e usuário
POST	/api/categorias	Cadastra uma nova categoria	Sim	Admin
PUT	/api/categorias/:id	Atualiza uma categoria existente	Sim	Admin
DELETE	/api/categorias/:id	Remove uma categoria	Sim	Admin

Jogos

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/jogos	Lista os jogos do catálogo, com filtros	Sim	Admin e usuário
GET	/api/jogos/:id	Busca um jogo pelo identificador	Sim	Admin e usuário
POST	/api/jogos	Cadastra um novo jogo (com upload de imagem)	Sim	Admin
PUT	/api/jogos/:id	Atualiza um jogo existente (com upload de imagem)	Sim	Admin
PATCH	/api/jogos/:id/estoque	Ajusta manualmente o estoque de um jogo	Sim	Admin
DELETE	/api/jogos/:id	Remove um jogo	Sim	Admin

Empréstimos e reservas

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/emprestimos	Lista os empréstimos cadastrados, com filtros	Sim	Admin
GET	/api/emprestimos/:id	Busca um empréstimo pelo identificador	Sim	Admin
POST	/api/emprestimos	Cria um empréstimo diretamente pelo administrador	Sim	Admin
PUT	/api/emprestimos/:id	Atualiza um empréstimo (datas, status, observações)	Sim	Admin
POST	/api/emprestimos/:id/devolver	Registra a devolução de um empréstimo	Sim	Admin
DELETE	/api/emprestimos/:id	Remove um empréstimo	Sim	Admin
POST	/api/reservas	Cria uma reserva solicitada pelo usuário comum	Sim	Usuário comum (e admin)
GET	/api/minhas-reservas	Lista as reservas do próprio usuário autenticado	Sim	Usuário comum (e admin)

Dashboard

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/dashboard/resumo	Retorna os indicadores do dashboard administrativo	Sim	Admin

Relatórios

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/relatorios/jogos/pdf	Gera o relatório de jogos em PDF	Sim	Admin
GET	/api/relatorios/emprestimos/pdf	Gera o relatório de empréstimos em PDF	Sim	Admin

Importação e exportação

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/exportacao/:entidade	Exporta os registros de uma entidade em JSON	Sim	Admin
POST	/api/importacao/:entidade	Importa registros de uma entidade a partir de um arquivo JSON	Sim	Admin

Logs

Método	Endpoint	Descrição	Requer login	Perfil
GET	/api/logs	Lista os logs registrados no sistema, com filtros	Sim	Admin
GET	/api/logs/exportar/xml	Exporta os logs filtrados em formato XML	Sim	Admin

As entidades aceitas pelas rotas de exportação e importação em JSON são usuarios, categorias, jogos e empréstimos, conforme implementado no ExportacaoDAO e no ExportacaoService. A rota DELETE /api/minhas-reservas/:id é a única ação de gestão de reserva disponível diretamente ao usuário comum, restrita ao cancelamento de reservas que ainda estejam pendentes e pertençam ao próprio usuário autenticado; as demais operações sobre empréstimos e reservas (aprovação, edição e devolução) são restritas ao administrador, conforme descrito na seção 4 deste documento.

19 CONSIDERAÇÕES FINAIS

O desenvolvimento do ProGames permitiu aplicar, em um projeto único, boa parte dos conceitos trabalhados na disciplina de Programação para Internet. O backend foi estruturado seguindo o padrão MVC, complementado pela camada de Service e pelo padrão DAO, com as interfaces IDAO, IService e IController utilizadas para padronizar o comportamento esperado de cada camada. O uso de middlewares permitiu concentrar, em um único lugar, preocupações como autenticação via JWT, verificação de permissão por perfil de usuário, validação de dados e tratamento de erros.

O sistema também trabalhou com dois modelos de banco de dados: o MySQL, utilizado para os dados operacionais da locadora, com tabelas relacionadas por chaves estrangeiras e consultas que utilizam filtros e junções; e o MongoDB, utilizado para o registro de logs,

aproveitando a flexibilidade de um banco de documentos para armazenar eventos de auditoria com estrutura variável.

Além do CRUD básico das entidades do sistema, o projeto incluiu funcionalidades adicionais como geração de relatórios em PDF, exportação e importação de dados em JSON, exportação de logs em XML e um dashboard com indicadores calculados a partir de consultas SQL e apresentados em gráficos com Chart.js. Esse conjunto de funcionalidades exigiu a integração entre frontend e backend por meio de uma API REST, consolidando, na prática, os principais tópicos estudados ao longo da disciplina.